NeuRoute: Logit-Guided Neural Routing for Billion-Scale Vector Search with Sub-Hour Index Construction

Xingqiao Wang
University of Arkansas at Little Rock
xwang1@ualr.edu

Zi Wang
University of Arkansas at Little Rock
zwang@ualr.edu

Xiaowei Xu*
University of Arkansas at Little Rock
xwxu@ualr.edu

ABSTRACT

Building approximate nearest neighbor (ANN) indexes at billion scale is often dominated by expensive global clustering or graph construction, making time-to-index a first-order systems concern. We present *NeuRoute*, a learned hashing index that turns short binary codes into an effective routing primitive for large-scale vector search. *NeuRoute* trains a lightweight neural network encoder with a selective similarity-preserving objective to produce well-balanced binary addresses. During construction, *NeuRoute* organizes vectors into buckets by their codes and performs *bucket-local* clustering in the encoder's low-dimensional space to form centroids. At query time, *NeuRoute* exploits the encoder logits as an uncertainty signal: it uses deviation-to-threshold scores to prioritize uncertain-bit perturbations for query-adaptive multi-bucket probing, scores *bucket-local* centroids by their distances to the query to form a compact candidate cluster set, and applies centroid-stage gating with heap-quality-driven early stopping to prune low-value clusters before exact refinement. On billion-scale benchmarks, *NeuRoute* achieves strong accuracy–throughput trade-offs with fast index construction: on BigANN-1B it reaches 90.3% Recall@10 at 2,414 QPS and is 1.7× faster than OPQ+IVF-PQ (refine) at comparable accuracy, while completing end-to-end training+construction in under an hour on both BigANN-1B and Deep1B. These results show that logit-guided neural routing can make hashing competitive as a lightweight ANN indexing framework at billion scale.

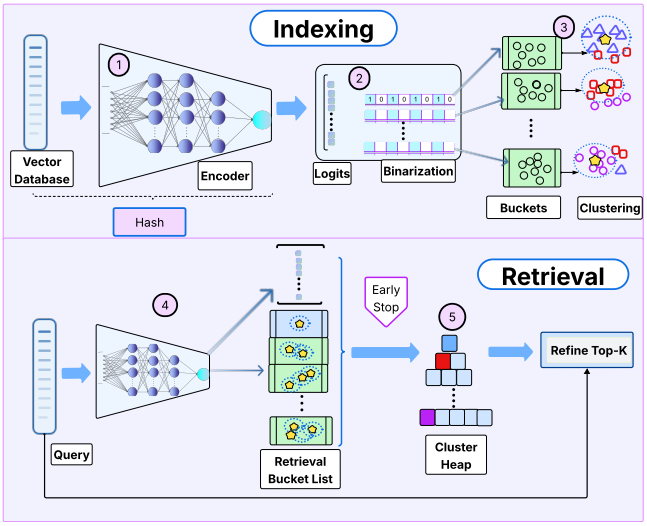

Figure 1: Overview of the *NeuRoute* pipeline (Steps 1–5). Steps 1–3: hash learning and index construction; Steps 4–5: logit-guided candidate generation with calibrated gating and early stopping before exact scoring.

PVLDB Reference Format:
Xingqiao Wang, Zi Wang, and Xiaowei Xu. NeuRoute: Logit-Guided Neural Routing for Billion-Scale Vector Search with Sub-Hour Index Construction. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/XingqiaoWang/NeuRoute/tree/main.

1 INTRODUCTION

Billion-scale similarity search is a cornerstone of modern data management and machine learning systems, supporting applications such as image retrieval, recommendation, deduplication, and large-scale embedding search [39, 40, 42]. To meet strict latency and memory constraints, practitioners rely on approximate nearest neighbor (ANN) indexes, with graph-based methods and disk-resident hybrids offering strong recall–latency trade-offs on large datasets [4, 18, 27, 43]. However, at billion scale these approaches often incur substantial system costs—large index footprints, expensive construction and tuning, and heavy preprocessing such as global clustering or graph building—which become first-order concerns under single-node resource budgets [18, 43, 50].

Hashing provides a complementary design point. Binary codes are compact, inexpensive to store, and fast to compute, enabling lightweight partitioning of massive databases into buckets (lists) that can be scanned efficiently [9, 12, 44]. Yet, despite extensive progress in learning better codes, hashing has struggled to become a dependable routing mechanism at billion scale under tight scan budgets [17, 50]. A core limitation is that many pipelines expand search using discrete Hamming-radius probing (e.g., radius 0/1/2/...) with weak prioritization, which provides limited control over where computation is spent [12, 29, 44]. Moreover, after binarization many methods discard pre-binarization activations (logits) that encode bit uncertainty—precisely the signal needed to prioritize probes and to terminate early when additional scanning is unlikely to improve results.

*Corresponding author. Email: xwxu@ualr.edu
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

In this work, we revisit hashing from a systems perspective and ask: **Can short binary codes become a dependable neural router for billion-scale retrieval under a strict scanning budget?** We propose **NeuRoute**, an unsupervised neural hashing framework designed explicitly for routing efficiency and budget-controlled candidate generation. Figure 1 sketches the end-to-end pipeline: Steps 1–2 learn a lightweight encoder that produces short binary addresses (§3.2); Step 3 builds a bucketized index and injects additional selectivity via *bucket-local* clustering in the learned low-dimensional space (§3.3); and at query time, Steps 4–5 perform logit-guided bucket enumeration and centroid-stage selection with calibrated gating and heap-quality-driven early stopping (§3.4, Algs. 1 and 2). Concretely, NeuRoute uses deviation-to-threshold scores derived from encoder logits to prioritize uncertain-bit perturbations for query-adaptive multi-bucket probing, then ranks bucket-local centroids by distance to form a compact candidate cluster set. A calibrated centroid-stage gate together with heap-quality-driven early stopping prunes low-value clusters, after which exact scoring in the original embedding space is used only for final top-k refinement.

NeuRoute is built around a simple systems observation: while short binary codes are attractive for compact routing and fast bucketization, hashing-only retrieval remains insufficient at billion scale under strict scan budgets. To make short codes useful for ANN without heavy offline pair mining or graph dependencies, NeuRoute trains a lightweight encoder with a selective similarity-preservation objective computed on-the-fly within mini-batches (§3.2), avoiding expensive offline neighbor-graph construction and integrating naturally into frequent re-indexing workflows. However, even with prioritized bucket enumeration, routing with short codes alone can still yield excessive refinement workload at practical recall levels. This motivates NeuRoute's second-stage structure: lightweight bucket-local clustering in the learned low-dimensional space (§3.3), which adds centroid organization within each bucket to support aggressive candidate reduction before exact refinement.

Importantly, this structure is both feasible and impactful at billion scale: bucket-local clustering completes in minutes as part of Add/Build (e.g., 380 s on BigANN-1B and 614 s on Deep1B-1B) while improving end-to-end throughput by up to 4.57× and reducing refinement candidates by up to 4.68× at matched Recall@10 (Table 9). To further keep query-time work budget-controlled, NeuRoute applies a calibrated centroid-stage gate together with a heap-quality–driven early-stop criterion (§3.4), yielding an additional 1.5×–1.7× QPS gain with negligible recall loss (Table 7). Finally, we implement logit-guided bucket enumeration as an efficient engineering primitive that operationalizes uncertainty-aware probing, allowing the system to concentrate effort on high-yield buckets and stop early when additional centroid evaluations no longer improve the retained set. Together, these components turn learned hashing from a strong but insufficient primitive into a fast-build, budget-controlled billion-scale retrieval system, enabling sub-hour end-to-end indexing time (0.82 h/0.93 h on BigANN-1B/Deep1B-1B) on a single node (§5.1, Table 3).

NeuRoute is most closely related to learned hashing and discrete representations, but it is designed for billion-scale routing: a lightweight training phase produces short binary codes for Hamming-space navigation and efficient candidate generation. Compared to graph- and disk-backed methods, NeuRoute reduces reliance on complex graph maintenance and storage/I/O-specific tuning. Compared to IVF–PQ pipelines, it avoids global coarse clustering over the full dataset and heavyweight quantizer training, relying instead on bucketized indexing with lightweight bucket-local clustering in low dimension. Overall, NeuRoute targets scalable retrieval with a simple index structure, fast time-to-index, and budget-controlled query-time compute through routing, centroid-stage filtering, and exact refinement.

Contributions. We make the following contributions:

- **Logit-guided neural routing with short codes.** We propose NeuRoute, an unsupervised MLP-based hashing framework that produces short, well-balanced binary addresses tailored for large-scale routing.
- **ANN-aligned, lightweight training objective.** We introduce an unsupervised objective that aligns the encoder with ANN neighborhoods while remaining lightweight to train.
- **Bucket-local clustering for centroid-stage filtering.** We introduce bucket-local clustering in the learned low-dimensional space to add lightweight centroid structure within buckets.
- **Billion-scale evaluation and fast time-to-index.** We conduct extensive experiments on two 1B-scale benchmarks (BigANN-1B and Deep1B-1B) and a high-dimensional setting, demonstrating strong Recall@10–throughput trade-offs and sub-hour end-to-end rebuilds (training + Add/Build) on a single node.

2 RELATED WORK

Efficient management of high-dimensional data underpins applications in recommendation, multimedia retrieval, and large-scale biological and language modeling [42]. To meet the computational demands of similarity search at scale, vector databases and ANN systems have developed multiple indexing paradigms—clustering-based, quantization-based, graph-based, and hashing-based—yet persistent production challenges remain, including high index build cost, large index footprints, and limited scalability under strict memory and latency budgets [34, 35]. These constraints complicate system deployment and tuning, while bounded compute and I/O budgets limit candidate exploration, causing practical efficiency to degrade despite strong offline accuracy.

Quantization and IVF-based indexes (FAISS IVF–PQ / OPQ). FAISS provides a widely used family of inverted-file indexes (IVF, IVF–PQ, IVF–OPQ) with SIMD-optimized kernels and runtime tuning via `ParameterSpace` (e.g., `nprobe`) [7, 19]. IVF–PQ partitions the space using k-means and compresses residuals with product quantization, supporting efficient lookup-table–based asymmetric distance computation (ADC) [20]. OPQ learns a rotation to reduce quantization distortion at fixed code size [11]. These quantization-based pipelines provide strong recall–latency trade-offs, but can suffer from list imbalance and non-trivial training/build overheads at large scale. We also adopt state-of-the-art prebuilt IVF–PQ configurations released for the BigANN benchmark/competition, which reflect heavily tuned quantization baselines at billion scale [38, 39].

Graph-based ANN (HNSW, disk-backed hybrids, and GPU graphs). Graph-based indexes such as HNSW achieve high recall through hierarchical routing and small-world connectivity [8, 27]. Other graph families include Voronoi-style routing structures (e.g., HVS) [24] and degree-reduced traversable graphs (e.g., NSG) that aim to lower traversal cost while maintaining navigability [10]. For billion-scale deployments, disk-backed hybrids such as DiskANN and SPANN combine in-memory structures with SSD-aware graph storage and pruning to balance recall, latency, and I/O efficiency [4, 18]. Recent GPU-accelerated graph search (e.g., CAGRA) further improves throughput by exploiting parallel traversal and memory bandwidth on modern devices [32]. Despite their effectiveness, graph construction is computationally intensive and typically scales poorly with dataset size and dimensionality; moreover, achieving competitive performance often requires substantial memory/SSD resources and careful tuning.

Hashing and binary-code retrieval (LSH, ITQ, and learned hashing). Hashing maps high-dimensional vectors into compact binary codes to enable fast Hamming-space filtering and low-memory indexing [9, 22, 25]. Classical LSH provides probabilistic candidate generation, including SimHash [3] and p-stable LSH for ℓ_p distances [5], building on the original LSH framework [14, 15, 17, 28, 50]. For an overview of near-optimal hashing results for ANN, see [1].

These methods are simple and parallelizable, but often require multi-table designs or aggressive probing to achieve high recall in high dimensions, inflating candidate sets and query latency. Multi-Probe LSH ranks likely perturbations to probe multiple nearby buckets within a table [26], and Multi-Index Hashing accelerates multi-probe enumeration in Hamming space via substring indexing [30]. While related in spirit, we drive probing using learned logits with a calibrated margin δ and enforce a strict budget via gating and early stopping.

Data-dependent hashing methods, including Spectral Hashing and ITQ, learn compact codes that better align with local neighborhoods compared with random projections [12, 44], but are commonly evaluated at smaller scales than billion-vector benchmarks. FAISS also provides `IndexLSH`, a training-free SimHash implementation typically paired with exact re-ranking for competitive recall [7]. Learned hashing further explores neural encoders to produce binary codes; TBH is a representative learned hashing pipeline that combines neural encoding with compact code generation and serves as a strong learned baseline in our evaluation [37].

Learned index structures and learned quantization. Learned index structures replace static indexing components (e.g., trees, hashes, quantizers) with models that exploit data regularities. Kraska et al. [21] introduced this viewpoint, with follow-up systems such as ALEX [6] and LISA [23] demonstrating improved adaptivity in ordered/spatial settings. In vector retrieval, a parallel trend appears in learned quantization and discrete representation learning. Early work (e.g., DQN) jointly optimized representation learning and quantization for retrieval [2], while more recent methods use distillation or constrained clustering to learn compact codes for dense retrieval (e.g., Distill-VQ, contrastive distillation variants, RepCONC) [31, 46, 49].

Position of NeuRoute. NeuRoute is most closely related to learned hashing and discrete representations, but it is designed for billion-scale routing: a lightweight training phase produces compact binary codes for Hamming-space navigation and efficient candidate generation. Compared to graph- and disk-backed methods, NeuRoute reduces dependency on complex graph maintenance and storage/I/O-specific tuning. Compared to IVF–PQ pipelines, it avoids global coarse clustering and heavyweight quantizer training, relying instead on bucketized indexing with lightweight bucket-local clustering in low dimension. Overall, NeuRoute targets scalable retrieval with low index-structure overhead and fast time-to-index, while using exact refinement for final ranking.

3 METHODOLOGY

This section presents the proposed *NeuRoute* framework for large-scale similarity search in vector databases. We begin by formally defining the approximate k-nearest neighbor (ANN) search problem. We then detail the design of (1) a learnable *hash function*—including the model architecture, loss formulation, and binarization process—and (2) the *index construction and retrieval* mechanism. Finally, we provide a brief *time-complexity* analysis.

3.1 Problem Definition

Let $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^{N} \subset \mathbb{R}^{E_{\text{dim}}}$ denote a set of N input embeddings, where $\mathbf{x}_i$ represents the embedding of a sample in a high-dimensional space of dimension E_{dim}. Given a query $\mathbf{q} \in \mathbb{R}^{E_{\text{dim}}}$ and the Euclidean distance $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2$, the goal of approximate k-nearest neighbor search is to retrieve a subset

$$S_k(\mathbf{q}) \subset \mathcal{X}, \quad |S_k(\mathbf{q})| = k, \tag{1}$$

such that the retrieved elements are close approximations of the true nearest neighbors:

$$S_k(\mathbf{q}) \approx \operatorname*{argmin}_{S:|S|=k} \sum_{\mathbf{x}\in S} d(\mathbf{q}, \mathbf{x}). \tag{2}$$

To enable efficient routing-based retrieval at scale, *NeuRoute* learns encoder logits that approximately preserve local neighborhood structure; the resulting short binary codes support fast multi-bucket probing, followed by centroid-based cluster filtering and exact refinement in the original embedding space.

Figure 1 provides a roadmap of NeuRoute (Steps 1–5). Steps 1–2 learn the hash function: embeddings are mapped to logits by the encoder, and then binarized by median thresholding to obtain balanced binary addresses. Step 3 builds the bucketized index and performs in-bucket clustering in the latent space, while storing original embeddings separately for exact refinement. At query time, Step 4 enumerates candidate buckets around the base address using a logit-guided procedure, and Step 5 selects candidate clusters via centroid distances with calibrated gating and heap-quality-driven early stopping before exact scoring.

This interleaved routing–refinement pipeline yields bounded candidate sets, supports early termination when no further improvements occur, and achieves a well-balanced trade-off among retrieval speed, memory efficiency, and search accuracy.

3.2 Hash Function

The *NeuRoute* framework efficiently transforms high-dimensional input embeddings into compact binary codes for rapid similarity search. This process involves two key transformations: (1) mapping vectors from the embedding space to a low-dimensional latent space via a neural encoder, and (2) binarizing the latent representations to produce discrete addresses in the Hamming space.

3.2.1 Encoder Definitions. At the core of *NeuRoute* lies an encoder architecture that compresses high-dimensional embeddings into compact latent representations and preserves their similarity. The encoder $E(\cdot)$ is formally defined as:

$$\boldsymbol{\ell} = E(\mathbf{X}), \quad \boldsymbol{\ell} = [\boldsymbol{\ell}_1, \boldsymbol{\ell}_2, \ldots, \boldsymbol{\ell}_N] \in \mathbb{R}^{N \times L_{\text{dim}}} \tag{3}$$

where $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N] \in \mathbb{R}^{N \times E_{\text{dim}}}$ denotes the input embeddings, $\boldsymbol{\ell}$ the latent representations. Here, $L_{\text{dim}} \ll E_{\text{dim}}$ represents the dimension of the latent space.

The encoder E extracts the most salient features of the high-dimensional input while suppressing redundancy.

3.2.2 Loss Function Design. The *NeuRoute* framework employs a selective similarity preservation loss. Preserving complete similarity structure of the data is difficult as the number of sample is large. On the other hand, we need only to maintain the similarity structure for vectors in proximity for ANN search. Therefore, we proposed a novel selective similarity-preservation loss (L_{sim})

The selective similarity loss ensures that neighborhood relationships in the embedding space are preserved in the latent space. Two pairwise distance matrices are defined under Euclidean distance: $\mathbf{S}_{\text{emb}} \in \mathbb{R}^{N \times N}$ for the original embeddings and $\mathbf{S}_{\text{lat}} \in \mathbb{R}^{N \times N}$ for the latent representations. The loss measures their discrepancy over selected pairs, using a masking mechanism to focus on semantically meaningful relationships:

$$L_{\text{sim}} = \frac{1}{\text{sum}(\text{Mask})} \sum_{i=1}^{N} \sum_{j=1}^{N} \text{Mask}_{ij} \Big(\mathbf{S}_{\text{lat}}(i,j) - \gamma \cdot \mathbf{S}_{\text{emb}}(i,j) \Big)^2, \tag{4}$$

where $\text{sum}(\cdot)$ denotes the element-wise summation of the mask matrix and γ is a scaling coefficient. The pairwise distance matrices are defined as:

$$\mathbf{S}_{\text{emb}}(i,j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2, \tag{5}$$

$$\mathbf{S}_{\text{lat}}(i,j) = \|\boldsymbol{\ell}_i - \boldsymbol{\ell}_j\|_2. \tag{6}$$

Metric. Throughout this paper, we use Euclidean distance:

$$d(x,y) = \|x - y\|_2. \tag{7}$$

The binary mask $\text{Mask} \in \{0,1\}^{N \times N}$ identifies significant pairs based on similarity thresholds:

$$\text{Mask}(i,j) = \begin{cases} 1, & \mathbf{S}_{\text{emb}}(i,j) \le \tau_{\text{emb}} \ \text{ or } \ \mathbf{S}_{\text{lat}}(i,j) \le \gamma \cdot \tau_{\text{emb}}, \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

Here, τ_{emb} defines the similarity threshold controlling the model's sensitivity to meaningful relationships. This selective focus enables *NeuRoute* to emphasize informative embedding pairs while ignoring weakly correlated ones.

Our objective is not to preserve the global similarity structure, but to optimize for ANN: we only need the very most similar pairs to remain reliably similar. Therefore, L_{sim} enforces consistency between the latent similarity S_{lat} and the input-space similarity S_{emb} only on top-quantile (nearest-neighbor) pairs (the mask selects the most similar $\sim 0.5\%$), concentrating model capacity on the region that directly determines recall. In addition, the dual-threshold anti-collapse mechanism prevents non-neighbors from becoming "artificially similar" in latent space, which helps avoid bucket explosion and unnecessary candidates. The scatter plots show a clear monotonic alignment in the near-neighbor regime while remaining well-behaved outside it, indicating that the objective is directly aligned with ANN's core trade-off: high recall with a controlled candidate set size.

This loss also eliminates expensive data preparation (e.g., building large-scale pair lists, mining hard negatives, or running offline neighbor graphs): the mask and thresholds are computed on-the-fly from within-batch similarities. As a result, training integrates naturally into the pipeline and reduces preprocessing overhead, which is a key enabler for our ∼1-hour end-to-end indexing workflow.

3.2.3 Binarization. For each latent dimension j, a per-dimension threshold τ_j is defined as the median activation across all samples:

$$\tau_j = \text{median}_{i=1,\ldots,N} \ \ell_i[j], \qquad j = 1, \ldots, L_{\text{dim}}.$$

Each latent activation is then binarized independently:

$$a_i[j] = \begin{cases} 1, & \ell_i[j] \ge \tau_j, \\ 0, & \ell_i[j] < \tau_j, \end{cases} \qquad i = 1, \ldots, N. \tag{9}$$

Here, $\ell_i[j]$ denotes the j-th component of the latent vector for sample i, and $a_i[j]$ represents the corresponding binary bit. This median-threshold binarization encourages balanced bit distributions across the dataset, stabilizing hash-bucket allocation. In practice, we estimate the per-dimension median thresholds on the same subsampled base used for the bucket-shape check, incurring no additional data pass; the time breakdown is reported in Table 3.

Deviation scores for bucket enumeration. After binarization, we define a per-dimension deviation score for a query q that measures how far the activation is from the threshold:

$$\delta_q[j] = \left| \ell_q[j] - \tau_j \right|, \quad j = 1, \ldots, L_{\text{dim}}. \tag{10}$$

Intuitively, a larger $\delta_q[j]$ indicates a more stable bit decision (farther from the threshold), while a smaller $\delta_q[j]$ suggests higher uncertainty near the threshold. We use $\{\delta_q[j]\}$ to select informative dimensions (e.g., the L_{sel} least-stable dimensions with the smallest $\delta_q[j]$) and to assign bit-flip costs when enumerating candidate buckets around the query's base binary address (Section 3.4).

3.2.4 Distance Preservation Diagnostics. To better understand how the learned encoding reshapes geometry, we visualize the relationship between pairwise distances measured in the original space and in the encoded space. Figure 2 shows two panels based on pre-sampled paired vectors: pairs sampled via the encoded space (left) and pairs sampled via the base space (right). Across both sampling schemes, the encoded distances remain strongly monotonic with the base distances, while large-distance pairs exhibit mild saturation in the encoded space. This behavior is consistent with our objective of preserving relative similarity for near neighbors, and it supports stable heap-quality signals for early stopping.

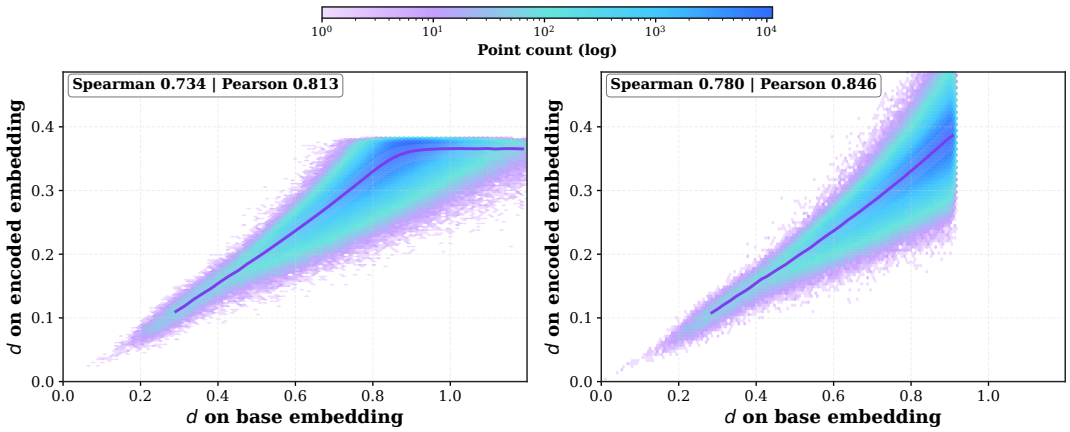

Figure 2: Pairwise distances in the base space versus the encoded space for paired vectors (squared for visualization; squaring is monotonic for nonnegative distances and does not change top-k ordering). Left: encoded-sampled pairs. Right: base-sampled pairs.

3.3 Index Construction

Figure 1 illustrates index construction. We start from the embedding space, where embeddings $\mathbf{X} \in \mathbb{R}^{N \times E_{\text{dim}}}$ are mapped by the encoder $E(\cdot)$ into a compact latent representation $\boldsymbol{\ell} = E(\mathbf{X}) \in \mathbb{R}^{N \times L_{\text{dim}}}$. Each latent vector is then binarized into a short address $\mathbf{A} \in \{0,1\}^{N \times L_{\text{dim}}}$, enabling bucketized indexing in Hamming space.

Bucket allocation. For each vector x_i, its binary address a_i determines the target bucket $b(a_i)$. We append the identifier i to the posting list of $b(a_i)$, and keep only non-empty buckets in the directory. The original embeddings $\{x_i\}$ are stored separately for exact top-k refinement.

In-bucket clustering in latent space. For each non-empty bucket b (i.e., the bucket indexed by an address u with a non-empty posting list), we collect the latent representations $\{\ell_i \mid i \in b\}$ of all vectors assigned to this bucket and perform clustering within the bucket in the latent space. We adopt a size-aware strategy based on bucket cardinality: very small buckets are not clustered and are treated as a single cluster, while medium and large buckets use different target cluster capacities. For a bucket B (with latent set $\{\ell_i \mid i \in B\}$), the number of clusters is determined adaptively as

$$K_B = \left\lceil \frac{|B|}{t} \right\rceil,$$

where t denotes the target cluster capacity. We first run k-means with K_B clusters on a sampled subset of $\{\ell_i\}$ with k-means++ initialization to obtain initial centroids, and then apply a limited number of Lloyd refinement passes on the full set $\{\ell_i \mid i \in B\}$ in that bucket. After clustering, each vector is assigned a local cluster label within its bucket, and a centroid representation is produced for each local cluster. Detail clustering results can be found in Appendix 7.

3.4 Retrieval Process

As shown in Figure 1 (Steps 4–5), NeuRoute performs *budgeted, prioritized* candidate generation before exact scoring. Given a query q, we first enumerate promising binary addresses (buckets) around its hash address under a bounded exploration budget (Step 4). We then process the resulting buckets in that order, evaluating their bucket-local cluster centroids and maintaining a bounded candidate set using calibrated distance gating and heap-quality-driven early stopping (Step 5). Finally, we expand the retained clusters into vector candidates and score them exactly in the original embedding space.

We use $d_{\text{cent}}(\cdot,\cdot)$ to denote the centroid-level distance used for candidate filtering, while exact scoring uses Euclidean distance $d(\cdot,\cdot)$ in the original embedding space, where $d(x,y) = \|x - y\|_2$.

Algorithm 1 Bucket Enumeration around the Base Address

Input: Query logit vector $\boldsymbol{\ell}_q \in \mathbb{R}^{L_{\text{dim}}}$; thresholds $\boldsymbol{\tau} \in \mathbb{R}^{L_{\text{dim}}}$; selection size L_{sel}; max flip radius $R_{\max}$; number of score bins N_{bins}; filter: max_score $S_{\max}$.

Output: Ordered bucket bins $\mathcal{B}_q[0..N_{\text{bins}}-1]$ (each is a list of bucket IDs).

$a_q[j] \leftarrow \mathbb{I}\big[\ell_q[j] \geq \tau_j\big] \quad \forall j = 1,\ldots,L_{\text{dim}}$
$b_0 \leftarrow \textsc{BucketId}(a_q)$
$\delta_q[j] \leftarrow \big|\ell_q[j] - \tau_j\big| \quad \forall j = 1,\ldots,L_{\text{dim}}$
$\mathcal{S} \leftarrow \textsc{TopKIndices}(\{\delta_q[j]\}_{j=1}^{L_{\text{dim}}}, L_{\text{sel}})$
Initialize empty bins $\mathcal{B}_q[0..N_{\text{bins}}-1]$
Append b_0 to $\mathcal{B}_q[0]$ ▷ score $s = 0$
for all $\Delta \subseteq \mathcal{S}$ with $1 \leq |\Delta| \leq R_{\max}$ **do**
  $b \leftarrow \textsc{ApplyFlips}(b_0, \Delta)$
  $s \leftarrow \sum_{j \in \Delta} \delta_q[j]$
  if $s \leq S_{\max}$ **then**
    $z \leftarrow \textsc{Bin}(s; N_{\text{bins}}, S_{\max})$
    Append b to $\mathcal{B}_q[z]$
  end if
end for
return $\mathcal{B}_q$

In our implementation, we bootstrap the calibration top-K lists by running the same end-to-end pipeline on a held-out set of calibration queries under a higher-recall setting (i.e., with an enlarged search budget/range), and use the resulting top-K outputs as supervision for calibration.

Bucket-score cutoff ($S_{\max}$). For each (q, target) pair in the top-K list, we compute the target bucket score using the same scoring function as in bucket enumeration. We then measure coverage as a function of a candidate threshold t: (i) the fraction of top-K pairs with score $\leq t$, and (ii) the fraction of queries whose entire top-K set is covered. We choose $S_{\max}$ to achieve high top-K bucket coverage while keeping the enumeration cost bounded.

Centroid gating margin ($\mathcal{M}$). For each query, we scan all centroids within the buckets that contain its top-K targets and record (1) the best-seen centroid distance x and (2) the additional margin y required so that the farthest top-K target would still pass a distance gate. We estimate a data-driven margin curve $\mathcal{M}(x)$ by grouping queries into equal-frequency (quantile) bins by x and taking a high quantile (e.g., 95th or 99th) of y within each bin, smoothing the result into a monotone piecewise-linear function. At inference time, the centroid-stage gate applies the threshold $\theta(x) = x + \mathcal{M}(x)$.

3.4.1 Bucket Enumeration (Step 4). Algorithm 1 summarizes Step 4. Given the query hash address $a_q = h(q)$, we apply a enumerator to generate and rank nearby binary addresses under a bounded enumeration budget. Enumerated addresses are grouped into priority bins and processed from high-yield to low-yield bins, enabling termination once additional enumeration no longer improves the retained candidates.

3.4.2 Centroid-Stage Selection with Calibrated Gating and Early Stop (Step 5). We process the enumerated buckets in increasing bin-score order (Alg. 1). Buckets that contain a single bucket-local cluster are added directly to $C^{\text{dir}}(q)$, since centroid-level selection offers limited benefit in this case.

For the remaining (non-singleton) buckets, we expand them to their bucket-local centroids and perform centroid-stage selection as summarized in Alg. 2: we compute centroid distances in batches, maintain a bounded top-K_c heap, and apply a calibrated gate $\theta = d_{\min} + M(d_{\min})$ to suppress low-value candidates. A heap-quality signal, controlled by a parameter α, triggers early stopping when additional centroid evaluations no longer improve the retained top-K_c set. Full implementation details are provided in Appendix 5.

The centroid stage returns $C^{\text{cent}}(q)$ (clusters retained in the heap), and the pre-refinement candidate set is

$$C(q) = C^{\text{dir}}(q) \ \cup \ C^{\text{cent}}(q).$$

Implementation. We implement the end-to-end retrieval pipeline in C++ for efficiency and reproducibility. The index is stored in a CSR-style layout to represent variable-length posting lists (e.g., bucket- and cluster-level inverted structures) compactly and enable sequential scans. At query time, we run the neural encoder via TorchScript (C++ API) to produce logits, and execute bucket/cluster enumeration and subsequent scanning entirely in C++ using vectorized distance kernels and multi-threading. Bucket enumeration is implemented with a DP-based generator to efficiently enumerate candidate addresses under the configured radius/score budget.

Algorithm 2 Centroid-Stage Cluster Enumeration with Calibrated Gating

Input: Query logits $\boldsymbol{\ell}_q$; ordered bucket bins $\mathcal{B}_q$ (Alg. 1); bucket-to-cluster map Φ; centroids $\{\boldsymbol{\mu}_c\}$; calibration margin $\mathcal{M}(\cdot)$; heap size K_c; batch cap B_{batch}.
Output: Candidate cluster set $C(q)$.

Initialize top-K_c heap H, and $d_{\min} \leftarrow +\infty$
for z in increasing order of bin score **do**
  $C_z \leftarrow \textsc{MapBucketsToClusters}(\mathcal{B}_q[z], \Phi)$
  for all batches $C_{batch} \subseteq C_z$ with $|C_{batch}| \leq B_{\text{batch}}$ **do**
    $\mathbf{d} \leftarrow \textsc{CentroidDistances}(\boldsymbol{\ell}_q, \{\boldsymbol{\mu}_c : c \in C_{batch}\})$
    $d_{\min} \leftarrow \min(d_{\min}, \min(\mathbf{d}))$
    $\theta \leftarrow d_{\min} + \mathcal{M}(d_{\min})$ ▷ calibrated gate
    $\textsc{KeepTopKUnderGate}(H, C_{batch}, \mathbf{d}, \theta)$
    if $\textsc{EarlyStop}(H, d_{\min})$ **then**
      break
    end if
  end for
  if $\textsc{EarlyStop}(H, d_{\min})$ **then**
    break
  end if
end for
return $C(q) \leftarrow \{c \mid (d, c) \in H\}$

3.5 System Complexity and Scalability Analysis

This section gives a stage-level characterization rather than a tight worst-case bound, since runtime is governed by adaptive budgets. For a query q, let B_q be the number of enumerated bucket addresses, C_q the number of visited centroids in latent space, and R_q the number of candidates entering exact refinement. The dominant query-time costs are centroid scoring in the latent space, approximately $O(C_q \cdot L_{\text{dim}})$, and refinement in the original embedding space, approximately $O(R_q \cdot E_{\text{dim}})$ under (squared) Euclidean distance. In practice, B_q, C_q, and R_q are controlled by the enumeration budget, calibrated gating, and heap-quality-driven early stopping.

For index construction, hash bucket assignment is linear in the database size N. Bucket-local clustering decomposes across buckets and is performed in the encoder's low-dimensional space, making it substantially cheaper than clustering in the original embedding dimension. Its practical cost is governed by (i) the latent dimension L_{dim}, (ii) the bucket size distribution (work per bucket), (iii) the per-bucket cluster budget, and (iv) the number of clustering iterations. Because the learned codes yield relatively balanced buckets, per-bucket workloads remain small and highly parallelizable; in our 1B-scale setting, we build centroids for roughly 4 million buckets in under 10 minutes as part of the overall indexing pipeline.

Overall, NeuRoute combines learned hashing with budget-aware candidate generation: bucket enumeration generates candidates, centroid-stage gating prunes low-yield regions, and early stopping limits work before exact refinement, enabling controllable per-query budgets at scale.

4 EXPERIMENT

This section presents an evaluation of the proposed *NeuRoute* framework under realistic vector-database conditions, including high-dimensional embeddings and large-scale corpora.

Table 1: Overview of datasets used in our evaluation.

Dataset	Domain	Type	Size
High-dimensional, moderate-scale evaluation sets			
GLDv2 (aug.) [45]	Landmark Recognition	Image	9.39M
Moderate-scale subsets (for build/query baselines)			
BigANN-100M [38]	SIFT Features	Image	100M
DEEP1B-100M [48]	CNN Features	Image	100M
Billion-scale, moderate-dimensional evaluation sets			
BigANN-1B [38]	SIFT Features	Image	1B
Deep-1B [48]	CNN Features	Image	1B

4.1 Datasets

We evaluate *NeuRoute* in two regimes (i) high-dimensional, moderate-scale datasets and (ii) billion-scale, moderate-dimensional benchmarks—summarized in Table 1.

High-dimensional, moderate-scale. We evaluate a high-dimensional regime on GLDv2 using 1536D image embeddings (DINOv2). This setting tests robustness under high dimensionality and a multi-million-scale corpus, complementing the 96D/128D billion-scale benchmarks.

Billion-scale. For the billion-scale with moderate-dimensional standard benchmarks, we use BigANN-1B (hand-crafted SIFT; 128-D; 1B) and Deep-1B (CNN features; 96-D; 1B) from the NeurIPS'21 BigANN challenge [38, 39]. We follow the prescribed base/query splits for all evaluations (Table 1).

4.1.1 Dataset Preprocessing.

Data preparation and embeddings. We expand the effective size of the Google Landmarks v2 (GLDv2) *train*, *index*, and *test* splits with standard augmentations—random cropping, horizontal flipping, and color jitter—without changing the evaluation protocol. All image datasets are embedded with the *DINOv2 (giant)*[33] model.

BigANN specifics. BigANN vectors are stored as integers (typically `uint8`). To make them compatible with neural training and index construction, we apply a byte-scale normalization by dividing by 255 *only during encoder training*. For evaluation and ground-truth computation, k-NN search and distance calculations are performed on the *original* integer database. This preserves comparability with FAISS baselines while enabling stable model fitting on normalized inputs.

4.2 Experiment Environment Setup

Our experiments were conducted on a single host with an **AMD EPYC 7543** CPU (2 sockets × 32 physical cores; 64 physical cores total; SMT enabled with 128 hardware threads) and 1007 GiB RAM across 8 NUMA nodes (~126 GiB per node), together with one **NVIDIA A100-PCIE-40GB** GPU. All datasets and index artifacts were stored on a **Lustre** parallel file system mounted at **/scrfs**; the host also provides **1.6 TB** of local NVMe SSD, which is used when required by disk-backed baselines (e.g., DiskANN).

The GPU is used only for training learned hashing models (NeuRoute and TBH) using **PyTorch 1.13.1** (Python 3.10, Linux x86_64). Unless otherwise stated, all reported encoding, index construction, query-time retrieval, and evaluation are executed on the CPU.

For CPU baselines, we use a consistent multithreading setting of 24 threads whenever applicable (e.g., FAISS OpenMP threads, HNSW search threads, and DiskANN's T parameter), and we explicitly note any single-thread measurements or method-specific constraints.

4.3 Baselines

Baseline selection and evaluation knobs. We evaluate ANN baselines that are reproducible and feasible on a single node up to 1B scale. To keep comparisons interpretable, we sweep one primary query-time knob per family to produce Recall@10–QPS frontiers: `nprobe` for IVF-based indexes, `efSearch` for HNSW, search depth L for DiskANN, and a single probing-budget scalar for 22-bit hashing ($b = 22$). Fixed build/training configurations are summarized in Table 2. At 1B scale, methods requiring full-resident vectors or large in-memory graphs (e.g., IVF-Flat and HNSW) are omitted under our single-node memory/storage/wall-time budgets; instead, we report OPQ+IVF-PQ with bounded refinement as an accuracy-oriented, disk-feasible reference.

Baselines. Graph (CPU). We include FAISS `IndexHNSWFlat` with fixed build settings (M=32, `efConstruction= 200`) and sweep `efSearch` on GLDv2 and the 100M subsets. **Graph (GPU reference).** We additionally report CAGRA [32] on 80M subsets due to the 40 GB device-memory limit, treated as a GPU-only throughput reference. **Disk-based.** We include DiskANN [18] using the official implementation on local NVMe and sweep the search depth L. **Quantization.** We include FAISS IVF-Flat/IVF-PQ [19, 20] where feasible, train codebooks on a fixed subset, build by streaming adds, and sweep `nprobe`. **1B refinement reference.** On 1B, we also report OPQ+IVF-PQ with bounded exact reranking [11, 20] by sweeping $k_{\text{factor}} \in \{50, 100, 200\}$ for $k = 10$ and reporting end-to-end query time. **Hashing.** We include 22-bit hashing baselines (LSH, ITQ [13], TBH [37]) and sweep a single probing-budget scalar; we additionally report bucket-occupancy and margin–flip diagnostics.

4.4 Metrics

Recall. We evaluate retrieval completeness using *Recall@k* with relevance judged *only by unique IDs*. For query i, let $\mathcal{G}_i^K$ be the ground-truth top-K ID set obtained by exact search (excluding the query itself), and let $\mathcal{R}_i^k$ be the system's returned top-k ID set. The per-query recall is

$$r_i(k) \;=\; \frac{\left|\mathcal{R}_i^k \cap \mathcal{G}_i^K\right|}{\min\!\left(K,\ |\mathcal{G}_i^K|\right)}\,. \tag{11}$$

Averaging over n_q queries yields

$$\text{Recall@}k \;=\; \frac{1}{n_q}\sum_{i=1}^{n_q} r_i(k)\,. \tag{12}$$

Quality–Throughput. Throughout, we report **Recall@k vs.Query Per Second(QPS)** [7] to show the quality–throughput trade-off, where QPS $= n_q/T_{\text{wall}}$ (total number of queries divided by wall-clock time); In figures, points farther up and to the right are better. For a hardware-agnostic view, we also report and plot **Recall@k vs. average candidates per query**—the mean number of database

Table 2: Configurations used in our evaluation. Baselines use fixed settings. NeuRoute lists query-time routing parameters (ranges indicate multiple configs observed in logs).

Setting	Method	Configuration
GLDv2 (1536D)	IVF-Flat	`IVF49152`
GLDv2 (1536D)	IVF-PQ	`OPQ96,IVF49152,PQ96x8`
BigANN-100M (128D)	IVF-PQ	`IVF49152,PQ32x8`
Deep1B-100M (96D)	IVF-Flat	`IVF49152`
Deep1B-100M (96D)	IVF-PQ	`IVF49152,PQ24x8`
BigANN-1B (128D)	IVF-PQ	`IVF49152,PQ32x8`
Deep1B-1B (96D)	IVF-PQ	`IVF49152,PQ24x8`
BigANN-1B (128D)	OPQ+IVF-PQ (refine)	`OPQ,IVF65536,PQ32x8`
Deep1B-1B (96D)	OPQ+IVF-PQ (refine)	`OPQ,IVF65536,PQ16x8`
As available	HNSW	`IndexHNSWFlat` (M=32, `efC`=200)
GLDv2 (1536D)	DiskANN	R=90, L_{build}=100, B=50, M=200, T=24
BigANN-100M	DiskANN	R=100, L_{build}=100, B=50, M=80, T=24
Deep1B-100M	DiskANN	R=100, L_{build}=100, B=50, M=110, T=24
BigANN-1B	DiskANN	R=48, L_{build}=64, B=50, M=80, T=24
Deep1B-1B	DiskANN	R=90, L_{build}=100, B=20, M=300, T=24
As available	CAGRA (GPU)	degree=32, inter=64, itopk=128, iter=100
As available	LSH / ITQ / TBH	22-bit codes
GLDv2 (1536D)	NeuRoute	L_{sel}=12–13, $R_{\max}$=4–5, K_c=10–1000, $S_{\max}$=0.07–0.09
BigANN-100M (128D)	NeuRoute	L_{sel}=14–16, $R_{\max}$=5–6, K_c=20–5000, $S_{\max}$=0.24–0.41
Deep1B-100M (96D)	NeuRoute	L_{sel}=14–15, $R_{\max}$=5–7, K_c=10–5000, $S_{\max}$=0.25–0.39
BigANN-1B (128D)	NeuRoute	L_{sel}=16, $R_{\max}$=6–7, K_c=500–5000, $S_{\max}$=0.36
Deep1B-1B (96D)	NeuRoute	L_{sel}=16–17, $R_{\max}$=4–7, K_c=500–10000, $S_{\max}$=0.25–0.36

vectors that receive an exact distance computation during refinement (labeled *"average candidates"* in the figures). This quantity is proportional to the amount of work and is comparable across systems.

4.5 Experimental Setup and Design

We design evaluations to reflect practical large-scale vector search settings rather than toy benchmarks. Unless otherwise noted, we use Euclidean distance and fix k=10 across all experiments, matching common top-k retrieval usage in recommendation systems, document search, and RAG-style pipelines [16, 36, 41, 42, 47]. We evaluate on high-dimensional embeddings and large-scale corpora (up to billion-entry bases) under strict runtime budgets in a single-node setup.

Benchmarks. We evaluate NeuRoute on (i) billion-scale benchmarks: BigANN-1B and Deep1B-1B; (ii) medium-scale counterparts: BigANN-100M and Deep1B-100M; and (iii) a high-dimensional robustness benchmark: GLDv2 (10M, 1536-d). Unless otherwise stated, each dataset uses n_q = 10,000 queries, and all methods are evaluated in the same batched setting. Configurations for all baselines and NeuRoute are summarized in Table 2; unless stated otherwise, NeuRoute uses α_{front} = 0.5–0.7 for the heap-quality early-stopping trigger.

4.5.1 NeuRoute Model and Hyperparameters.

Architecture. NeuRoute uses a lightweight MLP encoder that progressively compresses the input embedding from E_{dim} to a compact latent logit vector of dimension L_{dim}. Each hidden layer follows Linear→BatchNorm→ReLU, while the final encoder layer is linear to preserve the latent logits used for binarization and routing. Model instantiations differ only in layer widths to match dataset dimensionality; the layer pattern and training objective remain the same. At retrieval time, NeuRoute runs a single forward pass of the encoder $E(\cdot)$ to obtain the latent vector.

Choosing L_{dim} (bucket-count trade-off). We choose the code length L_{dim} to control the number of buckets and balance two competing costs. If L_{dim} is too large, the address space becomes overly sparse and query-time bucket enumeration becomes expensive (e.g., more buckets to probe and/or larger effective radii to reach sufficient candidates). If L_{dim} is too small, buckets become too coarse, increasing per-bucket workload and making bucket-local clustering and refinement scans more expensive. In practice, we choose L_{dim} based on dataset scale to keep bucket sizes manageable while maintaining an affordable enumeration budget. In our experiments, we use L_{dim}=22 for the 1B-scale benchmarks, L_{dim}=20 for the 100M-scale setting, and L_{dim}=16 for GLDv2.

Training protocol. We train NeuRoute using a fixed 2M-vector subset per dataset. We use mini-batches of size 4096 and optimize with Adam (learning rate 2×10^{-4}) for 500 epochs on all datasets. Under this schedule, training typically stabilizes early; full training and validation curves are provided in the Appendix 2.

Objective and fixed hyperparameters. We optimize the selective similarity-preservation loss L_{sim}, which aligns local neighborhood structure between the original embedding space and the latent space. We fix the scaling coefficient $\gamma = 0.6$ across datasets, and additionally apply a dimensionality-reduction factor $\sqrt{L_{\text{dim}}/E_{\text{dim}}}$; i.e., the overall scaling is $\gamma\sqrt{L_{\text{dim}}/E_{\text{dim}}}$.

Selecting τ_{emb}. We choose the threshold τ_{emb} in a dataset-specific manner via batch subsampling. Concretely, within each mini-batch we compute the distribution of pairwise distances and set τ_{emb} so that only the closest 0.5% pairs satisfy $S_{\text{emb}}(i, j) \le \tau_{\text{emb}}$, focusing the similarity constraint on retrieval-relevant neighborhoods. The resulting τ_{emb} values are reported in the Appendix 1.

4.5.2 Ablations and Sensitivity.

Centroid-stage early stopping. We ablate heap-quality-driven early stopping at the centroid stage (Step 5) on BigANN-1B with n_q=10,000 queries. All retrieval hyperparameters are kept fixed (L_{sel}=16, L_{dim}=22, E_{dim}=128, and K_c=3000). We compare early stopping enabled vs. disabled under two operating points (A/B) and report Recall@10, throughput (QPS), and the refinement workload measured by candidates routed to exact refinement per query (`Cand./q`, in thousands). Results are reported in Table 7.

Sensitivity of τ_{emb} and L_{sim} masking. We study sensitivity to the similarity-selection threshold τ_{emb} (selected-pair percentage) and to the pair-selection mask used in L_{sim} during training.

On Deep1B-100M, we compute Pearson and Spearman correlations over ∼2M within-batch sampled pairs *per setting*, using the same sampling rule and pair budget for each $\tau_{\text{emb}} \in [0.1\%, 0.5\%]$. Pairs are sampled either by thresholding in the original embedding space (*Base-sampled*) or in the encoded space (*Encoded-sampled*); the two procedures are applied independently and therefore may yield different pair sets. We report correlations for the default masked objective and for the unmasked objective. Results are reported in Table 8.

Bucket-local clustering. We ablate bucket-local clustering while keeping all other components unchanged. When clustering is disabled, each bucket is treated as a single group (no bucket-local

Table 3: NeuRoute build-time breakdown (seconds). Add/Build = Index build + CSR build + Clustering + Other/I/O, and Total = Train + Add/Build. Index build performs full-base encoding. CSR build constructs CSR files. Clustering is bucket-local clustering. Other/I/O is residual overhead.

Dataset	Train (s) [GPU]	Index build (s) [CPU]	CSR build (s) [CPU]	Clustering (s) [CPU]	Other/I/O (s) [CPU]	Add/Build (s) [CPU]	Total (s)
BigANN-100M	1,197.56	49.96	40.78	43.00	53.26	187.00	1,384.56 (0.39 h)
Deep1B-100M	1,176.27	41.85	64.44	69.00	53.71	229.00	1,405.27 (0.39 h)
BigANN-1B	1,183.46	529.29	420.50	380.00	442.21	1,772.00	2,955.47 (0.82 h)
Deep1B-1B	1,124.43	368.00	903.36	614.00	326.64	2,212.00	3,336.43 (0.93 h)
GLDv2	2,471.15	84.78	51.27	44.00	56.96	237.00	2,708.15 (0.75 h)

centroids), removing the centroid stage and routing candidates directly to refinement. We evaluate this ablation on 1B-scale datasets (BigANN-1B and Deep1B-1B) with n_q=10,000 queries, and report Recall@10, QPS, and candidates routed to exact refinement per query (Cand./q, in thousands), under the same retrieval configuration. Results are reported in Table 9.

5 RESULTS AND DISCUSSION

This section evaluates *NeuRoute* in terms of retrieval quality (Recall@10), throughput (QPS), and search effort (average candidates per query), together with end-to-end index build cost (training + construction), under Euclidean distance metrics.

Key takeaways. (i) **Sub-hour end-to-end build at 1B:** NeuRoute builds in 0.82 h on BigANN-1B and 0.93 h on Deep1B-1B, while DiskANN requires 17.96 h and 36.47 h, respectively. (ii) **Competitive serving point at ∼90% Recall@10:** on BigANN-1B, NeuRoute reaches 90.3% Recall@10 at 2,414 QPS, comparable to DiskANN (90.4% at 2,744 QPS). (iii) **Robust across dimensions:** on GLDv2 (1536D), NeuRoute still builds in 0.75 h under the same 2M training budget. (iv) **Pure hashing is not a competitive ANN index at 22 bits:** under Hamming-only probing, hashing baselines either achieve low Recall@10 or become collision-dominated (e.g., TBH on Deep1B), whereas NeuRoute maintains high bucket utilization with small hotspots.

5.1 Index Building

Table 4 reports end-to-end build cost (Train + Add/Build). For each dataset, *Add/Build* starts from pre-downloaded base vectors available on the storage system and measures the wall-clock time to encode the full base set and construct the index artifacts. NeuRoute uses a fixed 2M-vector training subset for all datasets; for GLDv2, we match the same 2M training budget across all trainable methods. NeuRoute's trained encoder is lightweight: the saved model checkpoint is 40 KB–4 MB on disk, depending on the chosen encoder capacity (width/depth), latent dimension, and output code configuration. NeuRoute *Train* corresponds to encoder training on a single A100 GPU, whereas NeuRoute *Add/Build* is CPU-only. Unless otherwise stated, all CPU-side stages (including NeuRoute Add/Build and all CPU baselines) use 24 threads. For DiskANN, index build and search use local NVMe storage as required by the method. Dataset download and external preprocessing are excluded from timing.

Footprint accounting. We report the on-disk *serving footprint*, which consists of (i) index metadata used for routing and (ii) a vector payload used for distance computation (full vectors or compressed codes); the original dataset files are not counted. Most baselines package metadata and payload together in their default index artifacts (e.g., IVF–Flat/HNSW store full vectors; IVF–PQ/OPQ store PQ codes; DiskANN stores a graph together with a compressed vector representation). NeuRoute separates these components for transparency: the routing index is lightweight, while exact refinement requires access to the base vectors. We therefore report routing-only / routing+co-located refinement vectors in Table 4; the latter should be compared against baseline footprints under the same "metadata+payload" definition. Unless otherwise stated, the NeuRoute query-time results in §6 use the co-located refinement vectors for stable throughput; omitting them reads vectors from the dataset/external vector store.

Table 4: End-to-end build cost breakdown (Train + Add/Build), in hours (h). Serving footprint (GB, bytes/10^9) counts the on-disk serving artifacts (index metadata plus any vector payload stored by the method); the original dataset files are not counted. Serving footprint is reported as *routing-only* / *routing+co-located refinement vectors*. Peak RSS (GB) is the maximum resident memory during build.

Dataset	Method	Train (h)	Add/Build (h)	Total (h)	Serving footprint (GB)	Peak RSS (GB)
GLDv2	IVF-PQ	7.15	3.64	10.80	1.28	18.83
GLDv2	IVF-Flat	9.32	4.32	13.63	58.07	149.21
GLDv2	DiskANN	0.00	1.89	1.89	76.93	107.92
GLDv2	HNSW	0.00	1.79	1.79	56.11	100.84
GLDv2	**NeuRoute**	**0.69**	**0.07**	**0.75**	**0.24 / 53.98**	**59.38**
BigANN-100M	IVF-PQ	1.17	5.96	7.12	4.03	20.54
BigANN-100M	IVF-Flat	1.26	6.66	7.92	52.03	74.13
BigANN-100M	DiskANN	0.00	3.02	3.02	58.51	79.02
BigANN-100M	HNSW	0.00	5.94	5.94	73.03	107.13
BigANN-100M	**NeuRoute**	**0.33**	**0.05**	**0.39**	**0.78 / 12.70**	**13.97**
BigANN-1B	IVF-PQ	0.63	31.45	32.08	40.03	86.69
BigANN-1B	DiskANN	0.00	17.96	17.96	341.33	98.32
BigANN-1B	OPQ+IVF-PQ	3.16	7.55	10.72	40.03 / 477.00	569.43
BigANN-1B	**NeuRoute**	**0.33**	**0.49**	**0.82**	**7.68 / 126.88**	**139.57**
Deep1B-100M	IVF-PQ	1.02	5.21	6.23	3.22	31.16
Deep1B-100M	IVF-Flat	0.63	3.06	3.69	39.22	92.74
Deep1B-100M	DiskANN	0.00	3.98	3.98	81.92	104.65
Deep1B-100M	HNSW	0.00	6.76	6.76	61.11	100.82
Deep1B-100M	**NeuRoute**	**0.33**	**0.06**	**0.39**	**0.78 / 36.54**	**40.19**
Deep1B-1B	IVF-PQ	0.58	31.44	32.02	29.82	74.82
Deep1B-1B	DiskANN	0.00	36.47	36.47	819.20	425.26
Deep1B-1B	OPQ+IVF-PQ	3.71	6.79	10.51	24.03 / 358.00	419.32
Deep1B-1B	**NeuRoute**	**0.31**	**0.61**	**0.93**	**7.67 / 365.30**	**386.83**

Build-time decomposition. As shown in Table 3, NeuRoute Add/Build decomposes into index build (full-base encoding plus a lightweight

calibration/evaluation step), CSR file construction, bucket-local clustering, and residual I/O. Across datasets, refinement-cache construction and bucket-local clustering account for a substantial fraction of Add/Build (roughly 40–69%), while the calibration/evaluation portion of index build is lightweight relative to the other components.

Build-time advantage. Across datasets, NeuRoute substantially reduces end-to-end build time under matched training budgets. On GLDv2, NeuRoute builds in 0.75 h versus 10.80 h for IVF–PQ (≈14–15× faster). On 100M-scale benchmarks, NeuRoute builds in 0.39 h versus 6.23–7.12 h for IVF–PQ (≈16–18× faster). Most notably, at 1B scale NeuRoute completes end-to-end build in 0.82 h (BigANN-1B) and 0.93 h (Deep1B-1B), which is ≈22× faster than DiskANN on BigANN-1B and ≈39× faster on Deep1B-1B, while also being ≈11–13× faster than OPQ+IVF–PQ (Train+Add/Build).

6 RETRIEVAL RESULTS

6.1 Baseline Accuracy–Throughput Trade-offs

Figure 3 summarizes the accuracy–throughput envelopes of representative ANN indexes. Across datasets, IVF-based methods follow the standard pattern: very high throughput at low recall when probing few lists, followed by steep QPS degradation as recall increases. For example, `IVF-PQ` is extremely fast in the low-recall regime (e.g., 110,360 QPS at 24.7% recall on `BigANN-100M` and 43,534 QPS at 28.2% recall on `BigANN-1B`), but its maximum recall remains limited under a fixed code budget (e.g., 68.5% on `BigANN-100M` and 63.9% on `BigANN-1B`). In contrast, `IVF-Flat` can approach near-exact recall, but only by scanning substantially more candidates, which reduces throughput (e.g., ≈ 309 QPS at 99.1% recall on `BigANN-100M`, and ≈ 240 QPS at 99.2% recall on `GLDv2`).

Graph-based indexes dominate the near-saturated recall regime on the 100M-scale settings. On `BigANN-100M`, HNSW reaches near-perfect recall with strong throughput (e.g., 28,084 QPS at 99.8% recall), while DiskANN offers a different trade-off (e.g., ≈ 4,996 QPS at ≈ 98.2% recall). On `Deep1B-100M`, HNSW similarly reaches high recall with high throughput (e.g., 31,361 QPS at 99.0% recall), and DiskANN achieves ≈ 3,617 QPS at ≈ 98.1% recall. On `GLDv2`, DiskANN achieves high recall with strong throughput (e.g., 5,152 QPS at 99.4% recall), while HNSW can be tuned either for higher QPS at slightly lower recall (e.g., 8,534 QPS at 97.5%) or for near-perfect recall at lower throughput (e.g., 1,673 QPS at 99.9%).

6.2 Smaller-Scale and High-Dimensional Results

While graph-based methods set a strong upper bound on moderate-scale benchmarks, NeuRoute targets a complementary operating point: *mid-to-high recall with predictable, budgeted candidate generation and substantially lower end-to-end build cost* (Table 4).

100M-scale results. On `BigANN-100M`, NeuRoute reaches 88.6% Recall@10 at 7,565 QPS and can extend to 95.9% Recall@10 at 2,665 QPS under the same sweep, while building end-to-end in 0.39 hours (Train+Add/Build). On `Deep1B-100M`, NeuRoute reaches 88.6% Recall@10 at 2,559 QPS and extends to 94.0% Recall@10 at 1,054 QPS, also with a 0.39 hour end-to-end build. These results show that NeuRoute remains competitive in the mid-to-high recall regime on 100M-scale datasets while retaining a much lower build-time footprint than graph/quantization pipelines.

Table 5: GLDv2 (1536D) comparison under the same evaluation harness (n_q=10,000). For fairness, NeuRoute is evaluated using only the learned 22-bit codes (no bucket-local clustering). Extended hashing frontiers and bucket-structure diagnostics are provided in the supplementary material.

Method (22b)	Recall@10 (%)	QPS
LSH	37.55	82.94
ITQ	57.27	111.17
TBH	74.99	18.52
NeuRoute (no clustering)	93.86	111.40

High-dimensional `GLDv2` *(1536D).* On `GLDv2`, NeuRoute remains usable at high dimensionality: it reaches 72.4% Recall@10 at 4,251 QPS, and reaches 88.1% Recall@10 at 814 QPS; pushing recall further to 92.7% yields 425 QPS. Meanwhile, NeuRoute builds end-to-end in 0.75 hours on the same node (Table 4), demonstrating that the approach continues to function in modern high-dimensional embedding spaces, even though graph-based baselines can reach higher near-saturated recall on this dataset.

Pure Hashing at 22 Bits (Context). Although NeuRoute uses short binary addresses, it is *not* a pure hashing index: query processing interleaves budget-aware bucket routing and centroid-stage filtering before exact refinement (§3.4). For context, we evaluate standard 22-bit hashing baselines (LSH, ITQ, TBH) under *Hamming-only* probing with the same bit budget ($b = 22$, stored as 3 bytes/vector when bit-packed). All methods are evaluated under the same harness (same query set, distance metric, and exact refinement implementation). On the high-dimensional 10M-scale setting (GLDv2, 1536D), these pure-hashing baselines exhibit a weak recall–throughput trade-off under short codes. In this regime, NeuRoute achieves consistently better accuracy–throughput performance under the same 22-bit budget. Table 5 reports a compact comparison.

For fairness in this subsection, NeuRoute is evaluated using *only* the learned 22-bit codes (with bucket-local clustering disabled) to isolate the effect of short-code routing under the same bit budget. We omit extended hashing frontiers and bucket-structure diagnostics from the main paper to prioritize the 1B-scale comparisons against strong ANN baselines (DiskANN and quantization-based methods); full hashing results and diagnostics are provided in the supplementary material.

GPU-only reference (CAGRA). We additionally ran the GPU graph baseline CAGRA [32] as a throughput reference. In our environment, CAGRA is device-memory limited (A100 40GB) and can be built only up to 80M vectors, so we do not include it in the main 100M/1B Recall@10–QPS comparisons; the 80M results are reported in the Appendix 8.

6.3 Full Billion-Scale Results

The billion-scale datasets highlight the limitations of pure `IVF-PQ` at fixed code size and the importance of better candidate generation. On `BigANN-1B`, `IVF-PQ` tops out at 63.9% recall in our sweep, while

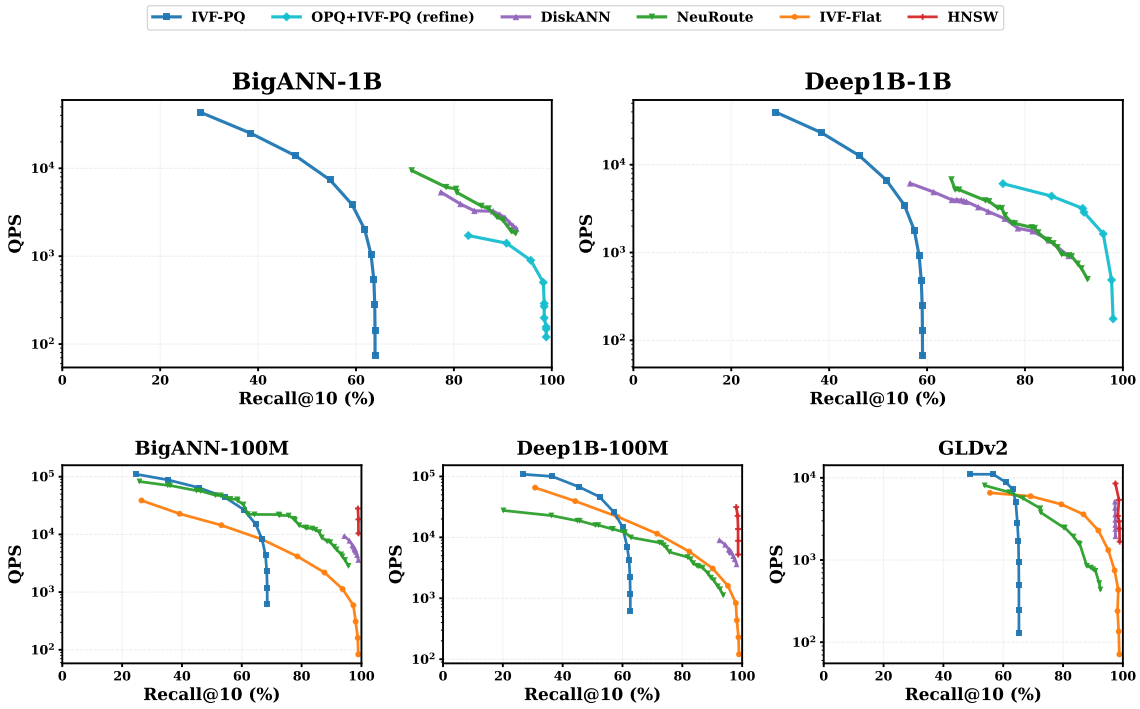

Figure 3: Recall@10–QPS trade-offs on `BigANN-1B`, `Deep1B-1B`, `BigANN-100M`, `Deep1B-100M`, and `GLDv2`. Each curve shows the Pareto frontier obtained by sweeping the main accuracy knob of each baseline (e.g., `nprobe` for IVF variants, `efSearch` for HNSW, and the search-budget parameters for DiskANN and rerank/refine variants). Higher is better.

DiskANN, NeuRoute, and reranked OPQ+IVF-PQ cover the high-recall regime. At an operating point near 90% Recall@10, NeuRoute sustains 2,414 QPS at 90.3% recall, comparable to DiskANN (2,744 QPS at 90.4% recall) and substantially faster than OPQ+IVF-PQ (refine) (1,406 QPS at 90.8% recall). Crucially, this operating point is achieved with dramatically lower build cost: NeuRoute builds in 0.82 hours versus 17.96 hours for DiskANN and 10.72 hours for OPQ+IVF-PQ (Train+Add/Build). DiskANN and NeuRoute reach similar ceilings in the low-90s recall range (DiskANN: 92.7% at 2,071 QPS; NeuRoute: 93.0% at 1,707 QPS), whereas OPQ+IVF-PQ (refine) is the only baseline here that pushes to ≈ 99.9% recall, at the cost of much lower throughput (≈ 121 QPS at 99.9% recall).

On `Deep1B-1B`, OPQ+IVF-PQ (refine) provides the strongest high-recall frontier in this set of runs, reaching 3,342 QPS at 91.2% recall and extending to 99.4% recall at 248 QPS. DiskANN reaches 88.9% Recall@10 at 915 QPS, while NeuRoute reaches 90.1% Recall@10 at 842 QPS; critically, NeuRoute builds in 0.93 hours versus DiskANN's 36.47 hours. Taken together, the two 1B datasets indicate that NeuRoute consistently reaches the low-90s recall regime with competitive throughput, while delivering *sub-hour* end-to-end build time at billion scale; reranked OPQ+IVF-PQ can extend to near-perfect recall when the additional refinement cost is acceptable.

Table 6: Billion-scale operating points *around* 90% Recall@10 (single-node). For each method, we report the highest QPS among configurations whose Recall@10 falls in a narrow band around 0.90 (here: $[0.88, 0.92]$). Build time is end-to-end Train+Add/Build from Table 4.

Dataset	Method	Recall@10	QPS	Build time (h)
`BigANN-1B`	DiskANN	0.9040	2,744	17.96
`BigANN-1B`	OPQ+IVF-PQ (refine)	0.9080	1,406	10.72
`BigANN-1B`	**NeuRoute**	0.9030	**2,414**	**0.82**
`Deep1B-1B`	DiskANN	0.8891	914.68	36.47
`Deep1B-1B`	OPQ+IVF-PQ (refine)	0.9120	3,342	10.51
`Deep1B-1B`	**NeuRoute**	0.9010	**842**	**0.93**

6.4 Ablation Studies

We ablate three components of *NeuRoute*: heap-quality-driven early stopping at query time, the similarity-selection threshold τ_{emb} and the pair-selection mask in L_{sim} during training, and bucket-local clustering in the retrieval pipeline.

Heap-quality-driven early stopping. We ablate heap-quality-driven early stopping on BigANN-1B with $n_q = 10{,}000$ queries. All retrieval hyperparameters are kept fixed: $L_{\text{sel}} = 16$, $L_{\text{dim}} = 22$, $E_{\text{dim}} = 128$, and $K_c = 3000$. We report throughput (QPS), Recall@10, and the

Table 7: Ablation of heap-quality-driven early stopping on BigANN-1B (n_q = 10,000).

Op.	Variant	Recall@10	QPS	Cand./q (k)
A	No early stop	0.90335	1440	249
A	Early stop	0.90260	2414	201
B	No early stop	0.90038	1682	230
B	Early stop	0.89965	2595	194

Table 8: Sensitivity to the training threshold τ_{emb} and the pair-selection mask on Deep1B-100M (database-sampled). Pearson/Spearman correlations are evaluated on a fixed set of 2,096,650 within-batch near-neighbor pairs, sampled as the top 0.5% most similar pairs separately in the base and encoded spaces (thus not necessarily the same pairs).

τ_{emb} (%)	Base-sampled pairs		Encoded-sampled pairs	
	Pearson	Spearman	Pearson	Spearman
0.1	0.8335	0.7722	0.8027	0.7362
0.2	0.8422	0.7789	0.8162	0.7429
0.3	0.8458	0.7813	0.8190	0.7457
0.5	0.8460	0.7800	0.8130	0.7430
0.5 (w/o mask, run-1)	0.8368	0.7535	0.5496	0.5617
N/A (w/o mask, run-2)	0.7391	0.6513	0.6701	0.5774

typical refinement workload in terms of candidates routed to exact refinement (`Cand./q` in thousands; computed from the query-level refinement statistics). Table 7 compares two operating points and shows a consistent trend: early stopping improves QPS by 1.5×–1.7× while reducing refined candidates by ∼16–19%, with a negligible absolute Recall@10 drop ($< 10^{-3}$).

Sensitivity to τ_{emb} and the pair-selection mask in L_{sim}. We study sensitivity to the top-quantile threshold τ_{emb} (fraction of selected within-batch pairs) and the pair-selection mask used in L_{sim}. We compute Pearson/Spearman correlations over 2,096,650 within-batch near-neighbor pairs, sampled separately in the original embedding space (*Base-sampled*) or in the encoded space (*Encoded-sampled*) using the same top-quantile rule (thus not necessarily the same pairs). As shown in Table 8, correlations remain stable when sweeping $\tau_{\text{emb}} \in [0.1\%, 0.5\%]$ under the default masked objective. In contrast, removing the latent-space mask (run-1) or removing masking entirely (run-2) substantially degrades correlations for encoded-sampled pairs, indicating that masking is important for filtering noisy pairs and preserving local neighborhood structure during training.

Bucket-local clustering. We ablate bucket-local clustering while keeping all other components unchanged. When clustering is disabled, each bucket is treated as a single group (no bucket-local centroids), so the centroid-stage is removed and candidates are routed directly to refinement. Table 9 shows that bucket-local clustering consistently reduces refinement load, leading to substantial throughput gains. On BigANN-1B, clustering improves QPS by 4.44× (2414 vs. 543.6), increases Recall@10 by +1.48 points (90.26% vs. 88.78%), and reduces candidates/query by 4.68× (201k vs. 940k). On Deep1B-1B, clustering improves QPS by 4.57× (756.9 vs. 165.5) and reduces candidates/query by 3.86× (351k vs. 1353k) with a marginal Recall@10 gain of +0.14 points (90.14% vs. 90.01%). Unless otherwise stated, we enable bucket-local clustering; we disable it in the 22-bit hashing comparison to isolate the effect of short-code routing under the same bit budget.

Table 9: Ablation of bucket-local clustering on 1B-scale datasets (n_q=10,000 queries). QPS is measured under the same retrieval configuration. Cand./q (k) is the number of vectors routed to exact refinement per query, reported in thousands.

Dataset	Variant	Recall@10	QPS	Cand./q (k)
`BigANN-1B`	w/ clustering	0.90260	2414	201
`BigANN-1B`	no clustering	0.88780	543.6	940
`Deep1B-1B`	w/ clustering	0.90144	756.9	351
`Deep1B-1B`	no clustering	0.90007	165.5	1353

Takeaways. (1) Centroid-stage early stopping is a safe efficiency knob: it yields a consistent 1.5×–1.7× QPS improvement by reducing refinement workload (∼16–19% fewer candidates) with a negligible Recall@10 drop ($< 10^{-3}$). (2) The L_{sim} training signal is robust to τ_{emb}: correlations remain stable when sweeping $\tau_{\text{emb}} \in [0.1\%, 0.5\%]$ under the masked objective. Masking is essential, especially for encoded-space sampling: removing the latent-space mask causes a large correlation collapse (e.g., Encoded-sampled Pearson/Spearman $\approx 0.81/0.74 \rightarrow 0.55/0.56$). (3) Bucket-local clustering is critical for throughput at billion scale: it cuts refinement load by 3.9×–4.7× and improves QPS by 4.4×–4.6×, while preserving (and sometimes improving) Recall@10 (e.g., +1.48 points on BigANN-1B and +0.14 on Deep1B-1B). Overall, NeuRoute's gains primarily come from reducing refinement work through centroid-stage structure, calibrated gating, and early termination.

7 CONCLUSION

This paper presented NeuRoute, an encoder-based hashing framework for scalable k-nearest-neighbor search in large vector databases. NeuRoute learns compact binary codes for fast, memory-efficient candidate generation, and combines logit-guided routing with calibrated gating and heap-quality-driven early stopping to achieve strong accuracy–throughput tradeoffs at billion scale. In our experiments, NeuRoute attains 90%+ Recall@10 on 1B-vector benchmarks while enabling sub-hour end-to-end builds on a single machine: encoder training is GPU-accelerated, whereas index construction and online retrieval are CPU-based.

NeuRoute also admits orthogonal system extensions, such as PQ-based reranking in the refinement stage, which may further improve the tradeoff curve. Future work will study how to support additional quantization variants (e.g., OPQ+PQ) within the same refinement interface. We also plan to harden NeuRoute for production deployment and release implementation details and evaluation artifacts to support reproducibility.

REFERENCES

[1] Alexandr Andoni and Piotr Indyk. 2008. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM* 51, 1 (Jan. 2008), 117–122. https://doi.org/10.1145/1327452.1327494

[2] Yue Cao, Mingsheng Long, Jianmin Wang, Han Zhu, and Qingfu Wen. 2016. Deep Quantization Network for Efficient Image Retrieval. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI '16)*. AAAI, 3457–3463. https://doi.org/10.1609/aaai.v30i1.10313

[3] Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing* (Montreal, Quebec, Canada) *(STOC '02)*. Association for Computing Machinery, New York, NY, USA, 380–388. https://doi.org/10.1145/509907.509965

[4] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 5199–5212. https://proceedings.neurips.cc/paper_files/paper/2021/file/299dc35e747eb77177d9cea10a802da2-Paper.pdf

[5] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry* (Brooklyn, New York, USA) *(SCG '04)*. Association for Computing Machinery, New York, NY, USA, 253–262. https://doi.org/10.1145/997817.997857

[6] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, David Lomet, and Tim Kraska. 2020. ALEX: An Updatable Adaptive Learned Index. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. ACM, 969–984. https://doi.org/10.1145/3318464.3389711

[7] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[8] Owen Pendrigh Elliott and Jesse Clark. 2024. The Impacts of Data, Ordering, and Intrinsic Dimensionality on Recall in Hierarchical Navigable Small Worlds. arXiv:2405.17813 [cs.IR] https://arxiv.org/abs/2405.17813

[9] Venice Erin Liong, Jiwen Lu, Gang Wang, Pierre Moulin, and Jie Zhou. 2015. Deep Hashing for Compact Binary Codes Learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[10] Cong Fu, Changxu Wang, and Deng Cai. 2017. Fast Approximate Nearest Neighbor Search With Navigating Spreading-out Graphs. *CoRR* abs/1707.00143 (2017). arXiv:1707.00143 http://arxiv.org/abs/1707.00143

[11] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. https://doi.org/10.1109/TPAMI.2013.240

[12] Yunchao Gong and Svetlana Lazebnik. 2011. Iterative quantization: A procrustean approach to learning binary codes. 817–824. https://doi.org/10.1109/CVPR.2011.5995432

[13] Yunchao Gong, Svetlana Lazebnik, Albert Gordo, and Florent Perronnin. 2013. Iterative Quantization: A Procrustean Approach to Learning Binary Codes for Large-Scale Image Retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, 12 (2013), 2916–2929. https://doi.org/10.1109/TPAMI.2012.193

[14] W. Han, H. Tang, and Y. Ye. 2022. Locality-Sensitive Hashing-Based k-Mer Clustering for Identification of Differential Microbial Markers Related to Host Phenotype. *Journal of Computational Biology* 29, 7 (Jul 2022), 738–751. https://doi.org/10.1089/cmb.2021.0640

[15] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing* (Dallas, Texas, USA) *(STOC '98)*. Association for Computing Machinery, New York, NY, USA, 604–613. https://doi.org/10.1145/276698.276876

[16] A. Iscen, T. Furon, V. Gripon, M. Rabbat, and H. Jégou. 2017. Memory vectors for similarity search in high-dimensional spaces. *IEEE Transactions on Big Data* 4, 4 (2017). https://ieeexplore.ieee.org/document/7870636

[17] Omid Jafari and Parth Nagarkar. 2021. Experimental Analysis of Locality Sensitive Hashing Techniques for High-Dimensional Approximate Nearest Neighbor Searches. In *Databases Theory and Applications*, Miao Qiao, Gottfried Vossen, Sen Wang, and Lei Li (Eds.). Springer International Publishing, Cham, 62–73.

[18] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf

[19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. Billion-scale similarity search with GPUs. arXiv:1702.08734 [cs.CV] https://arxiv.org/abs/1702.08734

[20] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[21] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. ACM, 489–504. https://doi.org/10.1145/3183713.3196909

[22] Hanjiang Lai, Yan Pan, Ye Liu, and Shuicheng Yan. 2015. Simultaneous feature learning and hash coding with deep neural networks. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE. https://doi.org/10.1109/cvpr.2015.7298947

[23] Pengfei Li, Hua Lu, Qian Zheng, Long Yang, and Gang Pan. 2020. LISA: A Learned Index Structure for Spatial Data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. ACM, 2119–2133. https://doi.org/10.1145/3318464.3389703

[24] Kejing Lu, Mineichi Kudo, Chuan Xiao 0001, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *PVLDB* 15, 2 (2021), 246–258. http://www.vldb.org/pvldb/vol15/p246-lu.pdf

[25] Xiao Luo, Haixin Wang, Daqing Wu, Chong Chen, Minghua Deng, Jianqiang Huang, and Xian-Sheng Hua. 2023. A Survey on Deep Hashing Methods. *ACM Trans. Knowl. Discov. Data* 17, 1, Article 15 (Feb. 2023), 50 pages. https://doi.org/10.1145/3532624

[26] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-probe LSH: efficient indexing for high-dimensional similarity search. In *Proceedings of the 33rd International Conference on Very Large Data Bases* (Vienna, Austria) *(VLDB '07)*. VLDB Endowment, 950–961.

[27] Yu Malkov and Dmitry Yashunin. 2016. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* PP (03 2016). https://doi.org/10.1109/TPAMI.2018.2889473

[28] Toan Nguyen Mau and Van-Nam Huynh. 2021. An LSH-based k-representatives clustering method for large categorical data. *Neurocomputing* 463 (2021), 29–44. https://doi.org/10.1016/j.neucom.2021.08.050

[29] Mohammad Norouzi, David J Fleet, and Russ R Salakhutdinov. 2012. Hamming Distance Metric Learning. In *Advances in Neural Information Processing Systems*, F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (Eds.), Vol. 25. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2012/file/59b90e1005a220e2ebc542eb9d950b1e-Paper.pdf

[30] Mohammad Norouzi, Ali Punjani, and David J. Fleet. 2014. Fast Exact Search in Hamming Space with Multi-Index Hashing. arXiv:1307.2982 [cs.CV] https://arxiv.org/abs/1307.2982

[31] James O'Neill and Sourav Dutta. 2023. Improved Vector Quantization for Dense Retrieval with Contrastive Distillation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*. ACM, 2072–2076. https://doi.org/10.1145/3539618.3592001

[32] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. arXiv:2308.15136 [cs.DS] https://arxiv.org/abs/2308.15136

[33] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2024. DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193 [cs.CV] https://arxiv.org/abs/2304.07193

[34] James Jie Pan, Jianguo Wang, and Guoliang Li. 2023. Survey of Vector Database Management Systems. arXiv:2310.14021 [cs.DB] https://arxiv.org/abs/2310.14021

[35] James Jie Pan, Jianguo Wang, and Guoliang Li. 2023. Survey of Vector Database Management Systems. arXiv:2310.14021 [cs.DB] https://arxiv.org/abs/2310.14021

[36] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. arXiv:2305.05065 [cs.IR] https://arxiv.org/abs/2305.05065

[37] Yuming Shen, Jie Qin, Jiaxin Chen, Mengyang Yu, Li Liu, Fan Zhu, Fumin Shen, and Ling Shao. 2020. Auto-Encoding Twin-Bottleneck Hashing. arXiv:2002.11930 [cs.CV] https://arxiv.org/abs/2002.11930

[38] Harsha Vardhan Simhadri and Big ANN Benchmarks Contributors. 2021. Big ANN Benchmarks: NeurIPS'21 T1/T2 README. GitHub repository. https://github.com/harsha-simhadri/big-ann-benchmarks Accessed: 2025-09-09.

[39] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (PMLR)*, Vol. 176. 177–189. https://proceedings.mlr.press/v176/simhadri22a.html

[40] T. Taipalus. 2024. Vector database management systems: Fundamental concepts, use-cases, and current challenges. *Cognitive Systems Research* 85 (2024), Article 101216. https://www.sciencedirect.com/science/article/pii/S1389041724000093

[41] Jizhe Wang, Pipei Huang, Huan Zhao, Zhibo Zhang, Binqiang Zhao, and Dik Lun Lee. 2018. Billion-scale Commodity Embedding for E-commerce Recommendation in Alibaba. arXiv:1803.02349 [cs.IR] https://arxiv.org/abs/1803.02349

[42] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. https://doi.org/10.1145/3448016.3457550

[43] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. arXiv:2101.12631 [cs.IR] https://arxiv.org/abs/2101.12631

[44] Yair Weiss, Antonio Torralba, and Rob Fergus. 2008. Spectral Hashing. In *Advances in Neural Information Processing Systems*, D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou (Eds.), Vol. 21. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2008/file/d58072be2820e8682c0a27c0518e805e-Paper.pdf

[45] Tobias Weyand, Andre Araujo, Bingyi Cao, and Jack Sim. 2020. Google Landmarks Dataset v2 - A Large-Scale Benchmark for Instance-Level Recognition and Retrieval. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2575–2584. https://github.com/cvdfoundation/google-landmark

[46] Shitao Xiao, Zheng Liu, Weihao Han, Jianjin Zhang, Defu Lian, Yeyun Gong, Qi Chen, Fan Yang, Hao Sun, Yingxia Shao, and Xing Xie. 2022. Distill-VQ: Learning Retrieval Oriented Vector Quantization by Distilling Knowledge from Dense Embeddings. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*. ACM, 1513–1523. https://doi.org/10.1145/3477495.3531799

[47] Xingrui Xie, Han Liu, Wenzhe Hou, and Hongbin Huang. 2023. A Brief Survey of Vector Databases. In *2023 9th International Conference on Big Data and Information Analytics (BigDIA)*. 364–371. https://doi.org/10.1109/BigDIA60676.2023.10429609

[48] Artem Babenko Yandex and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2055–2063. https://doi.org/10.1109/CVPR.2016.226

[49] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2023. Learning Discrete Representations via Constrained Clustering for Effective and Efficient Dense Retrieval (Extended Abstract). In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI '23)*. IJCAI, 6504–6508. https://doi.org/10.24963/ijcai.2023/728

[50] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (April 2023), 1979–1991. https://doi.org/10.14778/3594512.3594527